License: arXiv.org perpetual non-exclusive license
许可：arXiv.org 永久非独占许可
arXiv:1706.03762v7 [cs.CL] 02 Aug 2023
arXiv：1706.03762v7 [cs.CL] 2023 年 8 月 2 日

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.
只要提供适当的出处，谷歌特此授权仅用于新闻或学术作品，复制本文中的表格和图表。

Attention Is All You Need
注意力就是你所需要的全部

Ashish Vaswani
Google Brain
avaswani@google.com

Noam Shazeer[1]
Google Brain
noam@google.com

Niki Parmar[1]
Google Research
nikip@google.com

Jakob Uszkoreit[1]
Google Research
usz@google.com

Llion Jones[1]
Google Research
llion@google.com

Aidan N. Gomez[1]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser[1]
Google Brain
lukaszkaiser@google.com

Illia Polosukhin[1]
illia.polosukhin@gmail.com

Abstract 摘要

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

主流序列转导模型基于复杂的循环或卷积神经网络，这些网络包括编码器和解码器。性能最好的模型还通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构——变换器，仅基于注意力机制，完全摒弃重复和卷积。对两项机器翻译任务的实验显示，这些模型在质量上更优，同时更具并行性，且训练所需时间显著更少。我们的模型在 WMT 2014 英德翻译任务中达到了 28.4 BLEU，比现有最佳结果（包括合奏）提升了 2 个以上的 BLEU。在 WMT 2014 英法翻译任务中，我们的模型在八块 GPU 上训练 3.5 天后，建立了单模型最先进的 BLEU 得分 41.8，仅为文献中最佳模型训练成本的一小部分。我们证明，Transformer 通过成功应用于英语成分解析，无论是在大量且有限的训练数据下，都能很好地推广到其他任务。

1 Introduction 1 简介

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

循环神经网络，尤其是长短期记忆 [13] 和门控循环[7] 神经网络，已被牢固确立为序列建模和转导问题（如语言建模和机器翻译）领域的尖端方法 [35, 2, 5]。此后，众多努力不断推动循环语言模型和编码-解码器架构的边界 [38, 24, 15]。

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t, as a function of the previous hidden state h_{t-1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

循环模型通常沿输入和输出序列的符号位置进行计算。将位置对齐到计算时间的步骤，生成一系列隐藏状态 h_t，作为之前隐藏状态 h_{t-1} 和位置 t 输入的函数。这种本质上的顺序性排除了训练实例中的并行化，而在序列长度较长时，并行化变得尤为关键，因为内存限制限制了实例间的批量处理。近期工作通过分解技巧[21] 和条件计算[32] 显著提升了计算效率，同时在后者情况下也提升了模型性能。然而，顺序计算的基本约束依然存在。

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

注意力机制已成为各种任务中引人注目的序列建模和转导模型的重要组成部分，使得依赖关系的建模无需考虑输入或输出序列中的距离[2, 19]。然而，除了极少数情况 [27]，这些注意力机制通常与循环网络结合使用。

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

在本研究中，我们提出了变换器（Transformer），这是一种模型架构，摒弃了重复性，完全依靠注意力机制来绘制输入与输出之间的全局依赖关系。Transformer 实现了显著更多的并行化，经过在八块 P100 GPU 上训练短短十二小时后，即可达到新的翻译质量水平。

2 Background 2 背景

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [16], ByteNet [18] and ConvS2S [9], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [12]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

减少顺序计算的目标也构成了扩展神经 GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础，这些工具都以卷积神经网络为基本构件，并行计算所有输入和输出位置的隐藏表示。在这些模型中，连接两个任意输入或输出位置信号所需的操作次数随着位置间距离增加而增加，ConvS2S 是线性增长，而 ByteNet 则是对数增长。这使得学习远距离位置之间的依赖关系变得更加困难 [12]。在变换器中，这种现象被简化为恒定的运算次数，尽管由于注意力加权位置的平均，有效分辨率降低，这一效应我们用多头注意力（Multi-head Attention）来抵消，详见第 3.2 节。

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 27, 28, 22].

自注意，有时也称为内注意，是一种通过关联单个序列不同位置来计算序列表示的注意力机制。自我注意力已被成功应用于多种任务，包括阅读理解、抽象摘要、文本蕴涵以及学习与任务无关的句子表示 [4, 27, 28, 22]。

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [34].

端到端记忆网络基于循环注意力机制，而非序列对齐的重复出现，已被证明在简单语言问题回答和语言建模任务中表现良好 [34]。

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [17, 18] and [9].

据我们所知，变换器是首个完全依赖自关注计算输入和输出表示，而无需使用序列对齐 RNN 或卷积的转导模型。在接下来的章节中，我们将介绍变换器，激发自我关注，并讨论其相较于[17, 18] 和 [9] 等模型的优势。

3 Model Architecture
3 模型架构

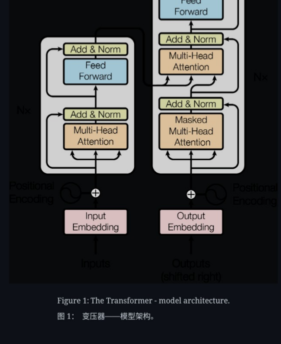


Figure 1: The Transformer - model architecture.
图 1：变压器——模型架构。

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 35]. Here, the encoder maps an input sequence of symbol representations $(x_1, ..., x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, ..., z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, ..., y_m)$ of symbols one element at a time. At each step the model is auto-regressive [10], consuming the previously generated symbols as additional input when generating the next.

大多数竞争性的神经序列转导模型具有编码器-解码器结构 [5, 2, 35]。这里，编码器将输入的符号表示序列映射 $(x_1, ..., x_n)$ 到连续表示序列 $\mathbf{z} = (z_1, ..., z_n)$。给定 $\mathbf{z}$，解码器随后逐个元素生成符号的输出序列 $(y_1, ..., y_m)$。每一步模型都是自回归的 [10]，生成下一步时会消耗之前生成的符号作为额外输入。

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

变换器遵循这一整体架构，采用叠层自注意力和点对点全连接层，分别用于编码器和解码器，分别见图 1 的左半部分和右半部分。

3.1 Encoder and Decoder Stacks
3.1 编码器和解码堆栈

Encoder: 编码器：

The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$, where $\mathrm{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{model} = 512$.

编码器由一组 $N = 6$ 个相同的层组成。每层有两个子层。第一种是多头自注意机制，第二种是简单的按位置完全连接的前向网络。我们在两个子层周围分别使用残差连接 [11]，随后进行层归一化 [1]。也就是说，每个子层的输出为 $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$，其中 $\mathrm{Sublayer}(x)$ 是子层自身实现的函数。为了促进这些残差连接，模型中的所有子层以及嵌入层都会产生维度为 $d_{model} = 512$ 的输出。

Decoder: 解码器：

The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

解码器还由一组 $N = 6$ 个相同的层组成。除了每个编码器层中的两个子层外，解码器还插入第三个子层，对编码器堆栈的输出进行多头关注。类似于编码器，我们在每个子层周围使用残差连接，然后进行层归一化。我们还修改了解码器堆栈中的自注意力子层，以防止位置关注后续位置。这种掩码，加上输出嵌入偏移一个位置，确保位置 i 的预测只能依赖于小于 i 位置的已知输出。

3.2 Attention 3.2 注意

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

注意函数可以描述为将查询和一组键值对映射到一个输出，其中查询、键、值和输出都是向量。输出以加权和的形式计算，每个值的权重由查询与对应键的兼容性函数计算。

3.2.1 Scaled Dot-Product Attention
3.2.1 缩放点积注意力

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

我们称我们特别关注点为"缩放点积注意力"（图 2）。输入由维度 d_k 的查询和键组成，维度值为 d_v。我们计算查询与所有键的点积，每个除以 $\sqrt{d_k}$，并应用 softmax 函数获得值的权重。

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as:

实际上，我们同时计算一组查询的注意力函数，这些查询被打包成矩阵 Q。键和值也被打包成矩阵 K 和 V。我们将输出矩阵计算为：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

最常用的两种注意力函数是加性注意力 [2] 和点积（乘法）注意力。点积注意力与我们的算法相同，唯一不同的是缩放因子为 $\frac{1}{\sqrt{d_k}}$。加法注意力通过具有单一隐藏层的前向网络计算兼容性函数。虽然两者在理论复杂度上相似，但点积注意力在实际操作中更快且更节省空间，因为它可以利用高度优化的矩阵乘法代码实现。

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients [4]. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

虽然对于较小的 d_k 值，两者表现相似，但加性注意力在较大 d_k 值下优于未缩放的点积注意力 [3]。我们怀疑对于较大的 d_k 值，点积会变得很大，从而将 softmax 函数推入梯度极小的区域 [4]。为抵消这一效应，我们将点积缩放为 $\frac{1}{\sqrt{d_k}}$。

3.2.2 Multi-Head Attention
3.2.2 多头注意力

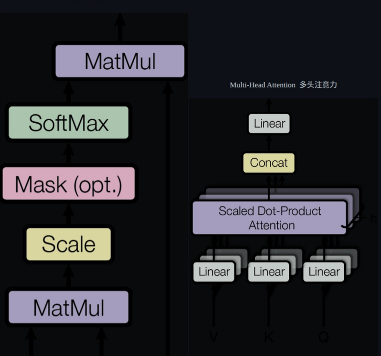


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.
图 2：（左）缩放点积注意力。（右）多头注意力由多个注意力层并行运行组成。

Instead of performing a single attention function with d_{model}-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

我们发现，与其用 d_{model} 维键、值和查询执行单一注意力函数，不如将查询、键和值线性投影 h 次，分别用不同的线性投影到 d_k、d_k 和 d_v 维度。在每个预测的查询、键和值版本上，我们并行执行注意力函数，得到 d_v 维输出值。这些数值被串联并再次投影，最终得到图 2 所示的最终数值。

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

多头注意力使模型能够共同关注来自不同位置表示子空间的信息。如果只有一个注意力，平均化会抑制这种效果。

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, ..., \mathrm{head}_h)W^O$$
$$\text{where } \mathrm{head}_i = \mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{model}}$.

其中投影为参数矩阵 $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$、$W_i^K \in \mathbb{R}^{d_{model} \times d_k}$、$W_i^V \in \mathbb{R}^{d_{model} \times d_v}$ 和 $W^O \in \mathbb{R}^{hd_v \times d_{model}}$。

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{model}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

在本作中，我们采用 $h = 8$ 个平行的注意力层，也就是"头"。对于这些，我们使用 $d_k = d_v = d_{model}/h = 64$。由于每个头的维数较小，总计算成本与单头全维注意力相近。

3.2.3 Applications of Attention in our Model
3.2.3 注意力在我们模型中的应用

The Transformer uses multi-head attention in three different ways:

变压器以三种不同方式使用多头注意力：

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [38, 2, 9].
  - 在"编码器-解码器注意"层中，查询来自前一层解码器层，内存键和值来自编码器的输出。这使得解码器中的每个位置都能覆盖输入序列中的所有位置。这模拟了序列对序列模型中典型的编码器-解码器注意力机制，如 [38, 2, 9]。
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
  - 编码器包含自注意层。在自注意层中，所有键、值和查询都来自同一处，即编码器中上一层的输出。编码器的每个位置都可以关注编码器上一层的所有位置。
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.
  - 同样，解码器中的自注意层允许解码器中的每个位置关注解码器中直到该位置的所有位置。我们需要防止解码器中的左向信息流，以保持自回归性质。我们在缩放点积注意力中实现这一点，方法是将软件输入中所有对应非法连接的值遮蔽（设置为 $-\infty$）。见图 2。

3.3 Position-wise Feed-Forward Networks
3.3 按位置划分的前馈网络

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

除了注意力子层外，我们编码器和解码器中的每一层都包含一个完全连接的前馈网络，该网络分别且完全相同地应用于每个位置。该变换包含两个线性变换，中间有 ReLU 激活。

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{model} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

虽然线性变换在不同位置上相同，但它们在层与层之间使用不同的参数。另一种描述方式是将其大小为 1 的两个卷积。输入和输出的维度为 $d_{model} = 512$，内层的维度为 $d_{ff} = 2048$。

3.4 Embeddings and Softmax
3.4 嵌入与 Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model}. We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [30]. In the embedding layers, we multiply those weights by $\sqrt{d_{model}}$.

与其他序列转导模型类似，我们使用学习嵌入将输入标记和输出标记转换为维度 d_{model} 的向量。我们还使用常用的学习线性变换和 softmax 函数将解码器输出转换为预测的下一个标记概率。在我们的模型中，两个嵌入层与 softmax 前的线性变换之间共享相同的权重矩阵，类似于 [30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{model}}$。

3.5 Positional Encoding
3.5 位置编码

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9].

由于我们的模型不包含递归和卷积，为了使模型能够利用序列的顺序，我们必须注入关于序列中符号相对或绝对位置的信息。为此，我们在编码器和解码器堆栈底部的输入嵌入中添加了"位置编码"。位置编码 d_{model} 与嵌入具有相同的维数，因此两者可以相加。位置编码有许多选择，有学习型和固定型[9]。

In this work, we use sine and cosine functions of different frequencies:

在本工作中，我们使用不同频率的正弦和余弦函数：

$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d_{model}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

其中 pos 是位置，i 是维度。也就是说，位置编码的每个维度都对应一个正弦波。波长形成从 2π 到 $10000 \cdot 2\pi$ 的几何级数。我们选择这个函数，是因为我们假设它能让模型轻松学会通过相对位置来关注，因为对于任意固定偏移 k，PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

我们还尝试使用学习位置嵌入 [9]，发现两种版本的结果几乎相同（见表 3 行（E））。我们选择正弦波版本，因为它可能允许模型推断出比训练中遇到的序列更长的序列长度。

4 Why Self-Attention
4 为什么要自我关注

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, ..., x_n)$ to another sequence of equal length $(z_1, ..., z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

本节将自注意层的各个方面与常用于将一个可变长度的符号表示序列 $(x_1, ..., x_n)$ 映射到另一个等长 $(z_1, ..., z_n)$ 序列的递归层和卷积层进行比较，例如 $x_i, z_i \in \mathbb{R}^d$ 典型序列转导编码器或解码器中的隐藏层。我们考虑三项动机来驱动我们使用自我关注。

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

其中之一是每层的总计算复杂度。另一个是可以并行化的计算量，以所需的最少顺序操作次数衡量。

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [12]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

第三个是网络中长距离依赖关系之间的路径长度。学习长距离依赖是许多序列转导任务中的关键挑战。影响学习此类依赖关系能力的一个关键因素是前进和后向信号在网络中必须经过的路径长度。输入和输出序列中任意位置组合之间的路径越短，学习长程依赖关系就越容易[12]。因此，我们还比较由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.
表 1：不同层类型的最大路径长度、每层复杂度以及最小的顺序操作次数。n 是序列长度，d 是表示维数，k 是卷积的核大小，以及 r 受限自注意下邻域的大小。

Layer Type 层类型	Complexity per Layer 每层复杂度	Sequential Operations 顺序运营	Maximum Path Length 最大路径长度
Self-Attention 自我关注	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent 常驻角色	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional 卷积	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted) 自我关注（受限）	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d, which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [38] and byte-pair [31] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

如表 1 所示，自注意层通过恒定数量的顺序执行操作连接所有位置，而重复层则需要 $O(n)$ 顺序操作。在计算复杂度方面，当序列长度 n 小于表示维度 d 时，自注意层比循环层更快，这在机器翻译中最先进的句子表示（如单词片段 [38] 和字节对[31]）中最为常见。为了提高涉及极长序列任务的计算性能，自我注意力可以限制为只考虑以相应输出位置为中心 r 的大小邻域。这将使最大路径长度增加到 $O(n/r)$。我们计划在未来的工作中进一步探讨这一方法。

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(log_k(n))$ in the case of dilated convolutions [18], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k. Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

单个卷积层的核宽度 $k < n$ 无法连接所有输入和输出位置对。在连续核（$O(log_k(n))$ 的情况下，需要堆叠 $O(n/k)$ 卷积层，或在扩张卷积 [18] 的情况下，增加网络中任意两个位置之间最长路径的长度。卷积层通常比循环层更昂贵，成本为 k 的倍数。然而，可分卷积 [6] 会显著降低复杂度，达到 $O(k \cdot n \cdot d + n \cdot d^2)$。然而，即使 $k = n$，可分卷积的复杂性也等于自关注层和点对点前馈层的组合，这也是我们在模型中采用的方法。

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

作为副产品，自我关注可能带来更可解释的模型。我们检查模型中的注意力分布，并在附录中展示和讨论示例。不仅个体注意力头清楚地学会执行不同的任务，许多人还表现出与句子句法和语义结构相关的行为。

5 Training 5 培训

This section describes the training regime for our models.

本节描述了我们模型的训练方案。

5.1 Training Data and Batching
5.1 训练数据与批处理

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [38]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

我们基于标准 WMT 2014 英德数据集进行训练，该数据集包含约 450 万句对。句子采用字节对编码[3]，其共享源-目标词汇约为 37000 个词元。对于英法，我们使用了显著更大的 WMT 2014 英法数据集，包含 3600 万句子，并将代币拆分为 32000 个词片词汇 [38]。句子对按近似序列长度批量在一起。每个训练批次包含一组句子对，约包含 25000 个源词元和 25000 个目标词元。

5.2 Hardware and Schedule
5.2 硬件与赛程

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models,(described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练模型。对于使用论文中描述的超参数的基础模型，每个训练步骤大约需要 0.4 秒。我们训练了基础模型，总共训练了 10 万步，也就是 12 小时。对于我们的大型模型（表 3 底部描述），步长为 1.0 秒。大型模型训练了 30 万步（3.5 天）。

5.3 Optimizer 5.3 优化器

We used the Adam optimizer [20] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

我们使用了带有 $\beta_1 = 0.9$、$\beta_2 = 0.98$ 和 $\epsilon = 10^{-9}$ 的 Adam 优化器 [20]。我们在训练过程中根据以下公式调整了学习率：

$$lrate = d_{model}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \tag{3}$$

This corresponds to increasing the learning rate linearly for the first $warmup_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup_steps = 4000$.

这对应于在最初 $warmup_steps$ 的训练步骤中线性增加学习率，之后按步数的平方根成反比降低学习率。我们使用了 $warmup_steps = 4000$。

5.4 Regularization
5.4 正则化

We employ three types of regularization during training:

我们在培训期间采用三种类型的正则化：

Residual Dropout 残余掉落

We apply dropout [33] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

我们在每个子层的输出中应用 dropout[33]，然后将其加入子层输入并归一化。此外，我们还对编码器和解码器堆栈中的嵌入和位置编码总和应用 dropout。对于基础模型，我们使用率， $P_{drop} = 0.1$

Label Smoothing 标签平滑

During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [36]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

培训期间，我们采用了价值的标签平滑 $\epsilon_{ls} = 0.1$ [36]。这会降低困惑度，因为模型会变得更不确定，但能提高准确率和 BLEU 分数。

6 Results 6 选举结果

6.1 Machine Translation
6.1 机器翻译

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.
表 2：Transformer 在英语到德语和英语到法语 newstest2014 测试中，比以往最先进型号的 BLEU 成绩更高，培训成本仅为其一小部分。

Model	BLEU		Training Cost (FLOPs) 培训费用（FLOPs）	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18] 字节网 [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32] 魔力之作 [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39] Deep-Att + PosUnk 乐团 [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38] GNMT + RL 合奏团 [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9] ConvS2S 集合 [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model) 变压器（基础型号）	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big) 变压器（大）	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

在 WMT 2014 英德翻译任务中，大变压器模型（表 2 中的 Transformer（大））比之前报告的最佳模型（包括集合）高出 2.0 BLEU，建立了新的先进 BLEU 得分 28.4。该模型的配置列于表 3 的底部。训练在 8 块 P100 显卡上花了 3.5 天。即便是我们的基础模型，也超越了所有已发布的模型和合奏，且训练成本仅为任何竞争模型的一小部分。

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{drop} = 0.1$, instead of 0.3.

在 WMT 2014 英法翻译任务中，我们的大模型获得了 BLEU 得分，41.0 优于所有先前发布的单一模型，且训练成本低于 1/4 之前最先进模型。训练成英译法语的 Transformer（大型）模型使用了中途率 $P_{drop} = 0.1$，而非 0.3。

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [38]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [38].

对于基础模型，我们使用了通过平均最后 5 个检查点得到的单一模型，这些检查点每隔 10 分钟写一个间隔写入。对于大型模型，我们对最后 20 个检查点进行了平均。我们使用了波束搜索，波束大小为 4 和长度惩罚为 $\alpha = 0.6$ [38]。这些超参数是在开发集上实验后选择的。我们在推断过程中将最大输出长度设置为输入长度 + 50，但尽可能提前终止 [38]。

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU [5].

表 2 总结了我们的结果，并比较了翻译质量和训练成本与文献中其他模型架构的比较。我们通过乘以训练时间、所用 GPU 数量以及每个 GPU 的持续单精度浮点容量估计浮点运算次数

6.2 Model Variations
6.2 型号变体

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.
表 3：变形金刚架构的变体。未公开的数值与基础型号相同。所有指标均基于英文转德文翻译开发集 newstest2013。列出的困惑是按字节对编码的每个字段计算的，不应与每个单词的困惑进行比较。

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps 列车步骤	PPL (dev) (开发者)	BLEU (dev) (开发者)	params 参数 $\times 10^6$
base 基地	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A) (A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B) (B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C) (C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D) (D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E) (E)	positional embedding instead of sinusoids 位置嵌入入代替正弦波									4.92	25.7	
big 大	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

为了评估 Transformer 不同组件的重要性，我们以不同方式调整了基础模型，测量开发集 newstest2013 中英德翻译性能的变化。我们使用了如前一节所述的波束搜索，但没有检查点平均。我们将这些结果展示在表 3 中。

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

在表 3 行（A）中，我们改变注意力头数及注意力键和值维度，保持计算量不变，详见第 3.2.2 节。虽然单头注意力比最佳设置差 0.9 BLEU，但头过多质量也会下降。

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [9], and observe nearly identical results to the base model.

在表 3 行（B）中，我们观察到减少注意力键大小 d_k 会影响模型质量。这表明确定兼容性并不容易，比点积更复杂的兼容性函数可能更有益。我们进一步观察到，在（C）和（D）行中，正如预期的那样，模型越大越好，而退出对避免过拟合非常有帮助。在第（E）行中，我们将正弦位置编码替换为学习到的位置嵌入 [9]，结果与基础模型几乎相同。

6.3 English Constituency Parsing
6.3 英格兰选区解析

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)
表 4：Transformer 很好地推广到英语成分解析（结果见《华尔街日报》第 23 节）

Parser 解析器	Training 培训	WSJ 23 F1
Vinyals & Kaiser el al. (2014) [37] Vinyals 和 Kaiser 等（2014）[37]	WSJ only, discriminative 仅限《华尔街日报》，带有歧视性	88.3
Petrov et al. (2006) [29] Petrov 等（2006）[29]	WSJ only, discriminative 仅限《华尔街日报》，带有歧视性	90.4
Zhu et al. (2013) [40] Zhu 等人（2013）[40]	WSJ only, discriminative 仅限《华尔街日报》，带有歧视性	90.4
Dyer et al. (2016) [8] Dyer 等（2016）[8]	WSJ only, discriminative 仅限《华尔街日报》，带有歧视性	91.7
Transformer (4 layers) 变压器（4层）	WSJ only, discriminative 仅限《华尔街日报》，带有歧视性	91.3
Zhu et al. (2013) [40] Zhu 等人（2013）[40]	semi-supervised 半监督式	91.3
Huang & Harper (2009) [14] 黄与哈珀（2009）[14]	semi-supervised 半监督式	91.3
McClosky et al. (2006) [26] McClosky 等人（2006）[26]	semi-supervised 半监督式	92.1
Vinyals & Kaiser el al. (2014) [37] Vinyals 和 Kaiser 等（2014）[37]	semi-supervised 半监督式	92.1
Transformer (4 layers) 变压器（4层）	semi-supervised 半监督式	92.7
Luong et al. (2015) [23] Luong 等人（2015）[23]	multi-task 多任务	93.0
Dyer et al. (2016) [8] Dyer 等（2016）[8]	generative 生成式	93.3

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [37].

为了评估 Transformer 是否能推广到其他任务，我们进行了英语成分解析实验。这项任务面临特殊挑战：输出受到强烈的结构约束，且远远超过输入长度。此外，RNN 序列对序列模型在小数据环境中未能获得最先进的结果 [37]。

We trained a 4-layer transformer with $d_{model} = 1024$ on the Wall Street Journal (WSJ) portion of the Penn Treebank [25], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkleyParser corpora from with approximately 17M sentences [37]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

我们在 Penn Treebank 的《华尔街日报》（WSJ）部分[25] 上训练了一个四层变压器 $d_{model} = 1024$，大约有 4 万句训练句子。我们还在半监督环境中训练它，使用了更大的高置信度语料库和 Berkley Parser 语料库，约 1700 万句子[37]。我们仅在《日报》设置中使用了 16K 词汇，半监督设置中使用了 32K 词汇。

We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we increased the maximum output length to input length + 300. We used a beam size of 21 and $\alpha = 0.3$ for both WSJ only and the semi-supervised setting.

我们只进行了少量实验，以选择掉线（注意力和残差）（第 5.4 节）、学习率和第 22 节发展集的束流大小，其他参数与英德基础翻译模型保持不变。在推断过程中，我们将最大输出长度增加到输入长度 + 300。我们用光束尺寸是 21 和 $\alpha = 0.3$，适用于仅限 WSJ 和半监督环境。

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar [8].

表 4 的结果显示，尽管缺乏针对任务的调整，我们的模型表现出乎意料地好，比之前报告的所有模型都更好，除了循环神经网络语法 [8]。

In contrast to RNN sequence-to-sequence models [37], the Transformer outperforms the BerkeleyParser [29] even when training only on the WSJ training set of 40K sentences.

与 RNN 序列对序列模型[37] 相比，Transformer 即使仅在 WSJ 训练集（4 万句子）上训练，表现也优于伯克利解析器 [29]。

7 Conclusion 7 结论

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

在这项工作中，我们提出了变换器，这是首个完全基于注意力的序列转导模型，用多头自注意取代了编码-解码器架构中最常用的递归层。

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

对于翻译任务，Transformer 的训练速度远快于基于递归层或卷积层的架构。在 WMT 2014 英德和 WMT 2014 英法翻译任务中，我们都达到了新的技术水平。在前者任务中，我们最好的模型甚至超过了之前报告的所有集合模型。

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

我们对基于注意力的模型的未来充满期待，并计划将其应用于其他任务。我们计划将变换器扩展到涉及文本以外输入输出模态的问题，并研究局部、受限的注意力机制，以高效处理图像、音频和视频等大输入和输出。让生成减少顺序性是我们的另一个研究目标。

The code we used to train and evaluate our models is available at https://github.com/tensorflow/tensor2tensor.

我们用来训练和评估模型的代码可在 https://github.com/tensorflow/tensor2tensor 获取。

Acknowledgements 致谢

We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

我们感谢 Nal Kalchbrenner 和 Stephan Gouws 提供的宝贵评论、纠正和激励。

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.

[8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.

[9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.

[10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.

[15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

[16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.

[17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.

[18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.

[19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.

[20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.

[22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.

[23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.

[24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

[25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

[26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.

[27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.

[28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.

[29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.

[30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.

[31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.

[35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.

[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.

[37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.

[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.

[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

Attention Visualizations
注意力可视化

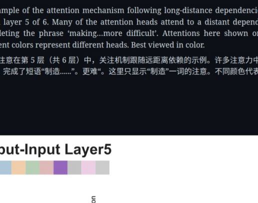


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.
图 3：编码器自注意在第 5 层（共 6 层）中，关注机制跟随远距离依赖的示例。许多注意力中心关注动词"制造"的远离依赖关系，完成了短语"制造……"。更难"。这里只显示"制造"一词的注意。不同颜色代表不同的头部。最好用彩色观看。

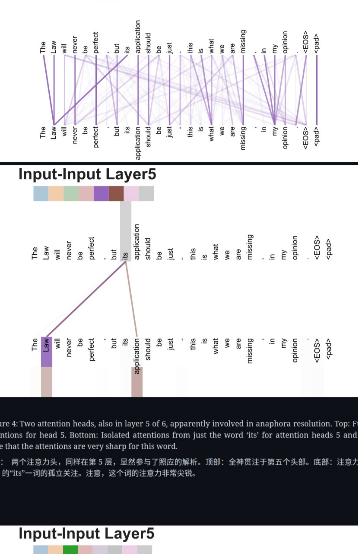


Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.
图 4：两个注意力头，同样在第 5 层，显然参与了照应的解析。顶部：全神贯注于第五个头部。底部：注意力 5 和 6 的"its"一词的孤立关注。注意，这个词的注意力非常尖锐。

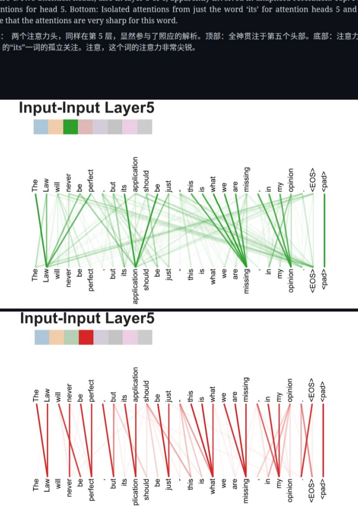


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.
图 5：许多注意力头表现出的行为似乎与句子结构有关。我们上文给出两个这样的例子，分别来自编码器自注意力的第 5 层（共 6 层）的两个不同头。头部显然学会了执行不同的任务。

Experimental support, please view the build logs for errors. Generated by LaTeXML.
请提供实验性支持。查看构建日志 为了错误。生成者 LaTeXML。

Instructions for reporting errors
错误报告说明

We are continuing to improve HTML versions of papers, and your feedback helps enhance accessibility and mobile support. To report errors in the HTML that will help us improve conversion and rendering, choose any of the methods listed below:

我们正在持续改进论文的 HTML 版本，您的反馈有助于提升无障碍和移动端支持。要报告有助于提升转换和渲染的 HTML 错误，请选择以下任一方法：

- Click the "Report Issue" button, located in the page header.
  点击页面页眉中的"举报问题"按钮。

Tip: You can select the relevant text first, to include it in your report.
提示： 你可以先选择相关文本，将其纳入报告中。

Our team has already identified the following issues. We appreciate your time reviewing and reporting rendering errors we may not have found yet. Your efforts will help us improve the HTML versions for all readers, because disability should not be a barrier to accessing research. Thank you for your continued support in championing open access for all.

我们的团队已经发现了以下问题。感谢您花时间审核并报告我们可能尚未发现的显示错误。你们的努力将帮助我们改进 HTML 版本，惠及所有读者，因为残疾不应成为访问研究的障碍。感谢您持续支持，推动所有人开放获取。

Have a free development cycle? Help support accessibility at arXiv! Our collaborators at LaTeXML maintain a list of packages that need conversion, and welcome developer contributions.

有免费的开发周期？帮助支持 arXiv 的无障碍功能！我们在 LaTeXML 的合作伙伴维护了一份需要转换的软件包列表，欢迎开发者的贡献。

Major funding support from 主要资金支持
SIMONS FOUNDATION